

DAY 2 · MODULE 2 · 40 MIN

Custom RAG on AI Search

When Foundry IQ isn't the right answer

Why this module

Module 1 introduced Foundry IQ — the managed knowledge layer. It's the right answer most of the time.

Sometimes it isn't. This module covers the tools you reach for when it isn't:

- Your source isn't a supported IQ connector
- You need control over indexing, chunking, or ranking that IQ doesn't yet expose
- You're already deep into an AI Search index with heavy customization
- You have latency SLAs below what IQ's pipeline delivers

Same platform under the hood — Azure AI Search. Different surface.

The classic RAG pattern — recap

Your app orchestrates three steps:

- Retrieve — send a query to AI Search, get back the top-K passages
- Augment — inject those passages into the prompt for your LLM
- Generate — LLM produces an answer grounded on what you retrieved

Simple. Fast. Fewer moving parts than agentic retrieval. You own the query pipeline end-to-end.

IQ hides steps 1 and 2 behind agentic retrieval; classic RAG puts you in the driver's seat.

Five RAG challenges — and how AI Search addresses them

Day 2 · Module 2

Challenge	AI Search answer
Query understanding — users' words rarely match your docs	Hybrid search + semantic ranker
Multi-source data — content is scattered	Indexers pull from 10+ sources; skills pipeline preprocesses
Token constraints — LLM context isn't infinite	Chunking + top-K + scoring profiles
Response time — users expect seconds	Millisecond queries; single-shot; you control retries
Security — private content stays private	Document-level trimming, filter-based ACLs, private endpoints

Module 1 solved these with IQ's LLM-driven pipeline. Classic RAG solves them with your own orchestration.

Content pipeline — indexers and skillsets

- Indexers — pull content from Azure Blob, OneLake, SharePoint, Cosmos, SQL, and other sources on a schedule; keep the index fresh
- Skillsets — apply transformations during indexing: OCR, image analysis, text splitting, embedding generation, custom skills
- Push API — when you'd rather pre-process content yourself and just load it in

Rule: prefer indexers + skillsets when your source is a supported connector. Push API for full control.

The three query modes

Mode	What it does	When
Keyword (full text)	Inverted index, BM25, exact-term matching	Structured content, exact-match, shared vocabulary
Vector	Similarity over embeddings — matches concepts	Conversational or vague queries, cross-language
Hybrid	Both in one query — merged and reranked	Almost always. Best recall.

Default recommendation for classic RAG: hybrid search + semantic ranker. Every Learn reference lands there.

Hybrid search — how it actually works

```
results = search_client.search(  
    search_text="How do I set up a Prompt agent?",  
    vector_queries=[VectorizableTextQuery(  
        text="How do I set up a Prompt agent?",  
        k_nearest_neighbors=5,  
        fields="contentVector",  
    )],  
    query_type=QueryType.SEMANTIC,  
    semantic_configuration_name="default",  
    top=10,  
)
```

- Same query, two lanes — keyword search on search_text; vector search on the same string embedded
- Reciprocal Rank Fusion (RRF) merges the two result lists
- Semantic ranker rescores the top 50 by learned relevance
- Returns the top-K to your app

Semantic ranker — the quiet weapon

Turns query understanding from 'keyword lookup' into 'did this passage actually answer the question?'

- Rescores your top 50 results using a learned model
- Extracts semantic captions — the sentence(s) that best answer the query
- Extracts semantic answers — extractive answer text, when the passage has one
- Same API call — set `query_type="semantic"` and pick a semantic configuration

Cost: pennies per query at typical volumes. Quality lift: usually 15–30% higher answer relevance. Almost always on.

Chunking strategy

The single biggest lever for retrieval quality.

Approach	Pros	Cons
Fixed-size (e.g., 500 tokens)	Simple, predictable	Cuts across semantic boundaries
Semantic (paragraph, section)	Chunks respect meaning	Uneven sizes; needs document structure
Sliding window with overlap	Preserves context across chunks	Duplicates content; more storage
Structured (per-record)	Perfect for tabular / DB content	N/A for unstructured text

Start with fixed-size 500-token chunks with 100-token overlap. Tune from there based on your eval scores (Module 3).

Embedding models

- text-embedding-3-small — most workshop scenarios; cheap, fast, 1536-dim, good multilingual
- text-embedding-3-large — better recall on hard queries; 3072-dim; more storage/compute
- Integrated vectorization in AI Search — pipe raw text through indexers; the service embeds it for you
- Match query-time embedding to index-time embedding (same model, same dimensions)

Building today? text-embedding-3-small with integrated vectorization: 90% of the way with 10% of the code.

Custom RAG in your MAF agent

Wire the search call into an MAF tool. The agent decides when to retrieve; your tool does the work.

```
from agent_framework import Agent, tool
from azure.search.documents import SearchClient

@tool(approval_mode="never_require")
def search_docs(query: str) -> str:
    """Search the docs index for content that answers a technical question.
    Use this whenever the user asks something specific about Foundry, MAF, or agents.
    Returns the top passages with citations."""
    results = search_client.search(
        search_text=query,
        vector_queries=[VectorizableTextQuery(text=query, k_nearest_neighbors=5, fields="contentVector")],
        query_type="semantic",
        semantic_configuration_name="default",
        top=5,
    )
    return format_results_with_citations(results)
```

agent = Agent(Client=LLMs, instructions="cite. Sources from search_docs", tools=[search_docs])

Security — document-level trimming

- Store user/group tags on each document at ingestion time
- Pass the caller's identity or group memberships as a filter at query time
- AI Search returns only documents the caller is authorized to see
- Same query, different results per user — the classic RAG equivalent of IQ's permission model

Trade-off vs. IQ: you build the ACL sync pipeline yourself. IQ syncs from SharePoint / OneLake for you.

When to choose classic RAG over Foundry IQ

Classic RAG

- You need GA features only — no preview surface for prod
- Existing orchestration or heavily-tuned index to preserve
- Fine-grained control over pipeline, ranking, filtering
- Very tight latency budget
- Source IQ doesn't yet connect to

Foundry IQ

- Source is a supported IQ connector
- Multiple agents will share the knowledge base
- Need permission-aware answers per caller without building it
- Want the agentic retrieval pipeline for free

Not either-or in real systems — often IQ for most content, classic RAG for a specialty index.

Sample repos to steal from

- [azure-search-openai-demo](#) — reference RAG chat app; agentic-retrieval-updated; ~15-min deploy via azd
- [azure-search-classic-rag](#) — classic RAG quickstarts in REST, Python, .NET, Java, JS, TS
- [azure-search-vector-samples](#) — vector-search patterns beyond the basics
- [microsoft/rag-time](#) — classic RAG time-journey scenarios

Repos are the source of truth for working code; Learn docs for concepts. Both apply.

Common traps

- Skipping the semantic ranker — 'it costs money' is a bad reason at typical volumes. Turn it on.
- Wrong chunk size — 100-token chunks lose context; 4000-token chunks blow past LLM context. 500 is safe.
- Mismatched embedding models — index with -small, query with -large. Vector queries return garbage. Match dimensions.
- Not evaluating retrieval separately — end-to-end tests can't tell retrieval bugs from generation bugs. Module 3 fixes this.
- No filter-based security — ships without ACLs, retrofit is painful.

Takeaways

- Classic RAG = you own the query pipeline. Same platform as IQ (Azure AI Search); different surface.
- Default recipe: hybrid search + semantic ranker. Rarely wrong.
- Chunk size is the biggest quality lever. Start at 500 tokens; tune with eval.
- Wire retrieval into a tool so the agent decides when to search; tool docstring is your control surface.
- Mix and match IQ and classic RAG in real systems.

Next: Evaluating retrieval — how you actually know if your knowledge layer works.